PRAKTIKUM 12
Filter IIR

NAMA: Ihsanta Zaki Sanjaya

KELAS: 2 D4 Teknik Komputer B

NRP: 3223600058

DOSEN: Dr Bima Sena Bayu Dewantara S.ST, M.T

MATA KULIAH: Praktikum Sinyal dan Sistem

TANGGAL: Rabu, 3 Desember 2024

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2024

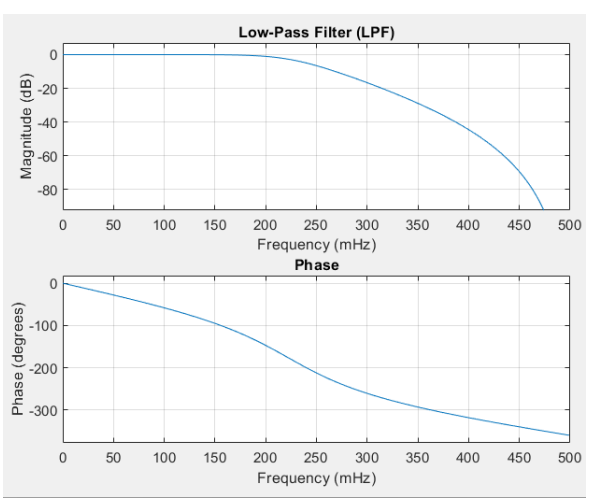
IIR Filter Prototipe

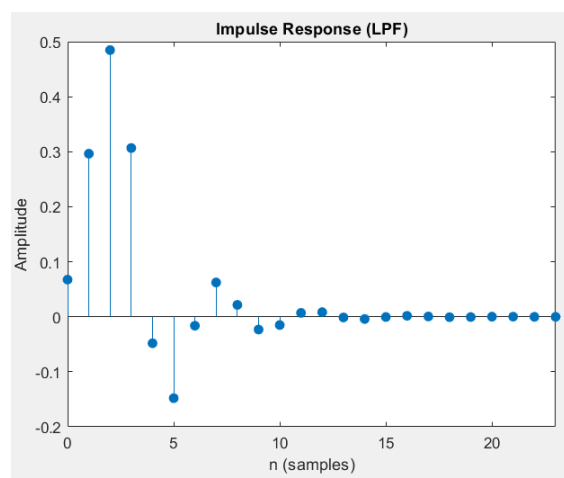
Program

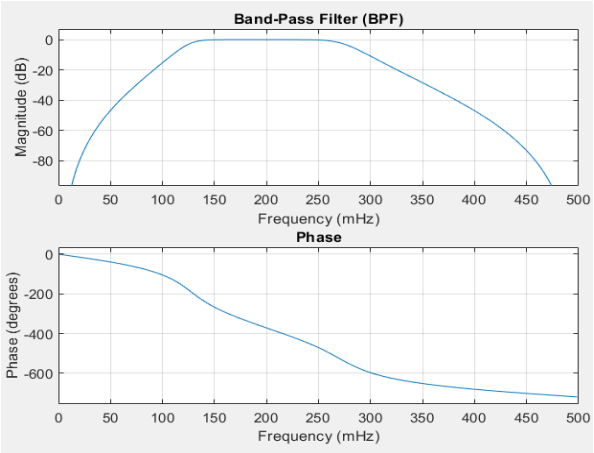
```
% Desain IIR Filter dengan Analog Butterworth Prototype

clc;
clear;
close all;

% Parameter desain
Fs = 1; % Frekuensi sampling normalisasi (1 Hz untuk normalisasi)
Wc_LPF = 0.45; % Cutoff frekuensi untuk LPF
Wc_HPF = 0.25; % Cutoff frekuensi untuk HPF
Wc_BPF = [0.25 0.55]; % Cutoff frekuensi untuk BPF
n = 4; % Orde filter

%% 1. Low-Pass Filter (LPF)
[bl, al] = butter(n, Wc_LPF, 'low'); % IIR LPF menggunakan Butterworth

% Respon LPF
figure;
freqz(bl, al, 1024, Fs); % Respon frekuensi
title('Low-Pass Filter (LPF)');

% Impulse response
figure;
impz(bl, al);
title('Impulse Response (LPF)');

% Analisis frekuensi cutoff
[H, f] = freqz(bl, al, 1024, Fs);
mag = 20*log10(abs(H));
idx_3dB = find(mag <= -3, 1);
f_cutoff_LPF = f(idx_3dB);

fprintf('Frek. cutoff (-3dB) LPF: %.3f Hz\n', f_cutoff_LPF);

%% 2. High-Pass Filter (HPF)
[bh, ah] = butter(n, Wc_HPF, 'high'); % IIR HPF menggunakan Butterworth

% Respon HPF
figure;
freqz(bh, ah, 1024, Fs); % Respon frekuensi
title('High-Pass Filter (HPF)');

% Impulse response
figure;
impz(bh, ah);
title('Impulse Response (HPF)');

% Analisis frekuensi cutoff
[H, f] = freqz(bh, ah, 1024, Fs);
mag = 20*log10(abs(H));
idx_3dB = find(mag <= -3, 1);
f_cutoff_HPF = f(idx_3dB);

fprintf('Frek. cutoff (-3dB) HPF: %.3f Hz\n', f_cutoff_HPF);

%% 3. Band-Pass Filter (BPF)
[bb, ab] = butter(n, Wc_BPF, 'bandpass'); % IIR BPF menggunakan Butterworth
```

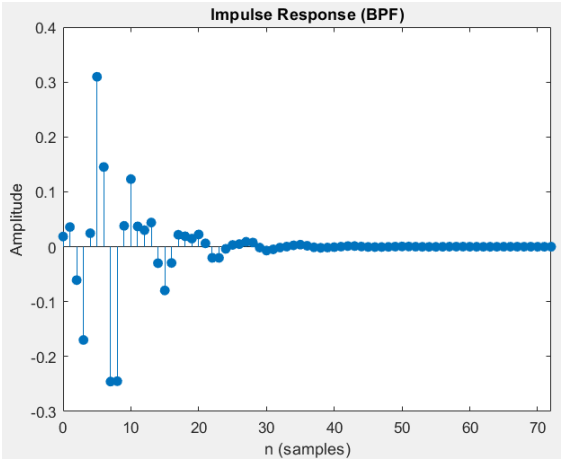
```
% Respon BPF
figure;
freqz(bb, ab, 1024, Fs); % Respon frekuensi
title('Band-Pass Filter (BPF)');

% Impulse response
figure;
impz(bb, ab);
title('Impulse Response (BPF)');

% Analisis frekuensi cutoff
[H, f] = freqz(bb, ab, 1024, Fs);
mag = 20*log10(abs(H));
idx_3dB_low = find(f <= Wc_BPF(1) & mag <= -3, 1, 'last');
idx_3dB_high = find(f >= Wc_BPF(2) & mag <= -3, 1, 'first');

f_cutoff_BPF_low = f(idx_3dB_low);
f_cutoff_BPF_high = f(idx_3dB_high);

fprintf('Frek. cutoff (-3dB) BPF: Low=%.3f Hz, High=%.3f Hz\n', f_cutoff_BPF_low,
f_cutoff_BPF_high);

%% 4. Plot Daerah Passband, Transition Band, Stopband
figure;
plot(f, mag, 'LineWidth', 1.5);
hold on;

% LPF
xline(f_cutoff_LPF, '--r', 'LineWidth', 1.5);

% HPF
xline(f_cutoff_HPF, '--g', 'LineWidth', 1.5);

% BPF
xline(f_cutoff_BPF_low, '--b', 'LineWidth', 1.5);
xline(f_cutoff_BPF_high, '--b', 'LineWidth', 1.5);

legend('Magnitude Response', 'LPF Cutoff (-3dB)', 'HPF Cutoff (-3dB)', ...
       'BPF Low Cutoff (-3dB)', 'BPF High Cutoff (-3dB)');
title('Filter Cutoff Frequencies');
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
grid on;
```

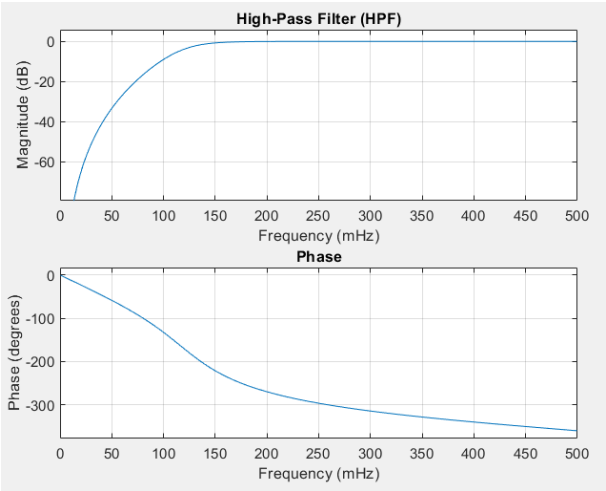
Output

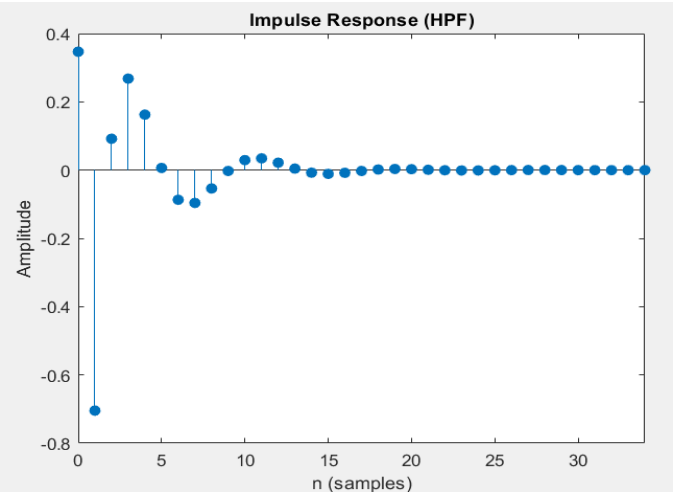


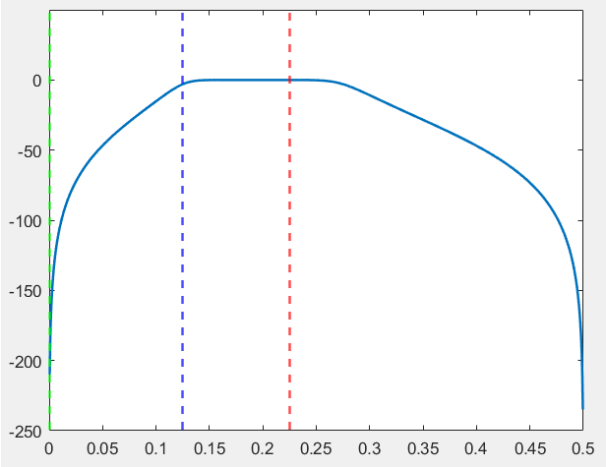












Garis Hijau: High Pass Filter

Garis Biru : Frekuensi Cutoff Low

Garis Merah : Frekuensi Cutoff High

Penjelasan:

Pada program ini menggunakan pendekatan prototipe analog (Butterworth) untuk mendesain filter IIR dengan tiga jenis: Low-Pass Filter (LPF), High-Pass Filter (HPF), dan Band-Pass Filter (BPF). Keunggulan metode ini adalah respons frekuensinya yang halus dengan roll-off yang konsisten. Program ini menghitung frekuensi cutoff secara otomatis berdasarkan analisis magnitude respons frekuensi pada titik -3 dB. Visualisasi meliputi respons frekuensi, respons impuls, serta perbandingan antara daerah passband dan stopband untuk setiap filter.

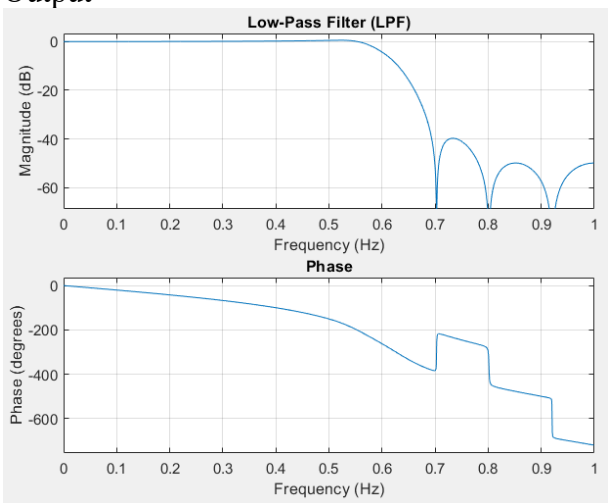
IIR Filter Direct

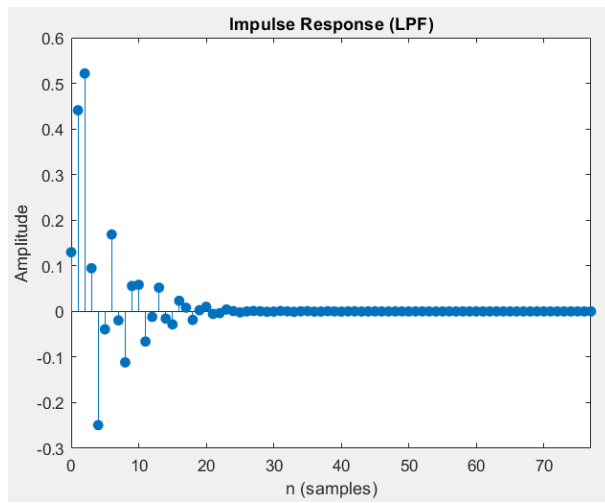
Program

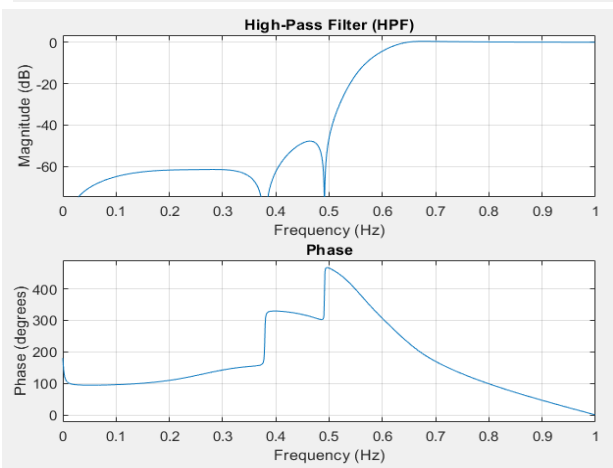
```
% Perancangan Filter IIR dengan Direct Design Method

clc;
clear;
close all;

% Parameter frekuensi normalisasi (0 - 1)
F = [0 0.15 0.25 0.35 0.45 0.55 0.65 0.75 0.85 1]; % Frekuensi pemetaan
N = 10; % Orde filter

%% 1. Low-Pass Filter (LPF)
% Vektor amplitudo untuk LPF
A_LPF = [1 1 1 1 1 1 0 0 0 0];

% Desain filter menggunakan yulewalk
[b_lpf, a_lpf] = yulewalk(N, F, A_LPF);

% Respon frekuensi LPF
figure;
freqz(b_lpf, a_lpf, 1024, 2); % Respon frekuensi
title('Low-Pass Filter (LPF)');

% Impulse response
figure;
impz(b_lpf, a_lpf);
title('Impulse Response (LPF)');

%% 2. High-Pass Filter (HPF)
% Vektor amplitudo untuk HPF
A_HPF = [0 0 0 0 0 0 1 1 1 1];

% Desain filter menggunakan yulewalk
[b_hpf, a_hpf] = yulewalk(N, F, A_HPF);

% Respon frekuensi HPF
figure;
freqz(b_hpf, a_hpf, 1024, 2); % Respon frekuensi
title('High-Pass Filter (HPF)');

% Impulse response
figure;
impz(b_hpf, a_hpf);
title('Impulse Response (HPF)');

%% 3. Band-Pass Filter (BPF)
% Vektor amplitudo untuk BPF
A_BPF = [0 0 1 1 1 1 0 0 0 0];

% Desain filter menggunakan yulewalk
[b_bpf, a_bpf] = yulewalk(N, F, A_BPF);

% Respon frekuensi BPF
figure;
freqz(b_bpf, a_bpf, 1024, 2); % Respon frekuensi
title('Band-Pass Filter (BPF)');
```

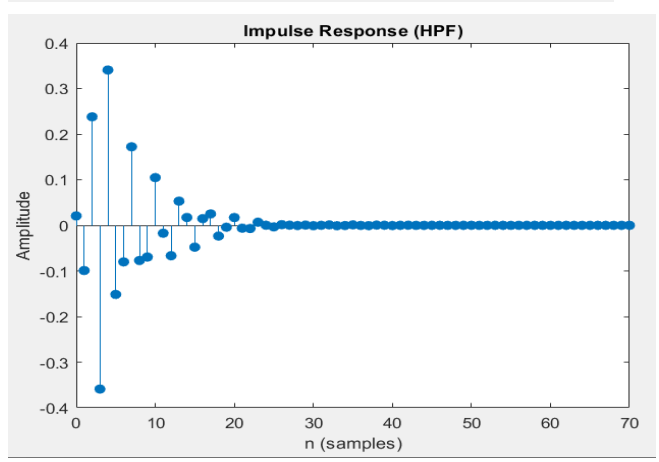
```
% Impulse response
figure;
impz(b_bpf, a_bpf);
title('Impulse Response (BPF)');

%% 4. Analisis Hasil
% Respon frekuensi gabungan untuk perbandingan
figure;
hold on;
[H_lpf, W] = freqz(b_lpf, a_lpf, 1024, 2);
[H_hpf, ~] = freqz(b_hpf, a_hpf, 1024, 2);
[H_bpf, ~] = freqz(b_bpf, a_bpf, 1024, 2);

plot(W, 20*log10(abs(H_lpf)), 'r', 'LineWidth', 1.5); % LPF
plot(W, 20*log10(abs(H_hpf)), 'g', 'LineWidth', 1.5); % HPF
plot(W, 20*log10(abs(H_bpf)), 'b', 'LineWidth', 1.5); % BPF

legend('LPF', 'HPF', 'BPF');
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title('Comparison of Filter Responses');
grid on;
hold off;
```

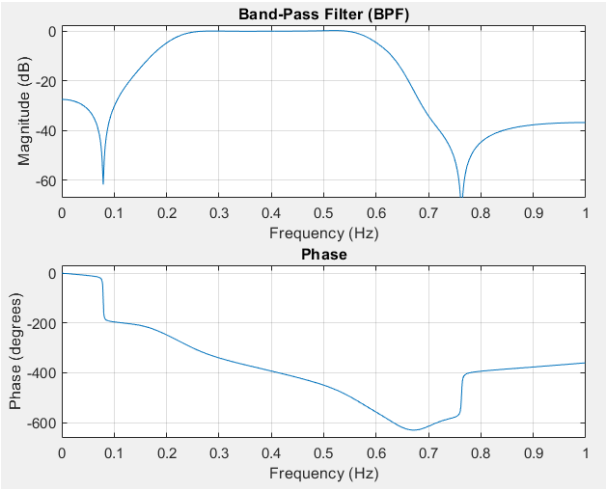
Output

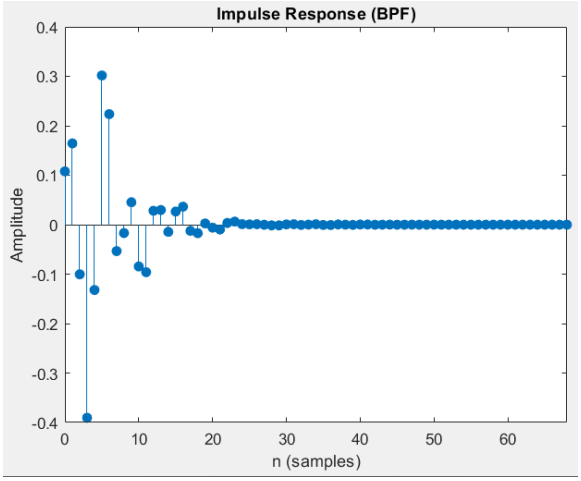


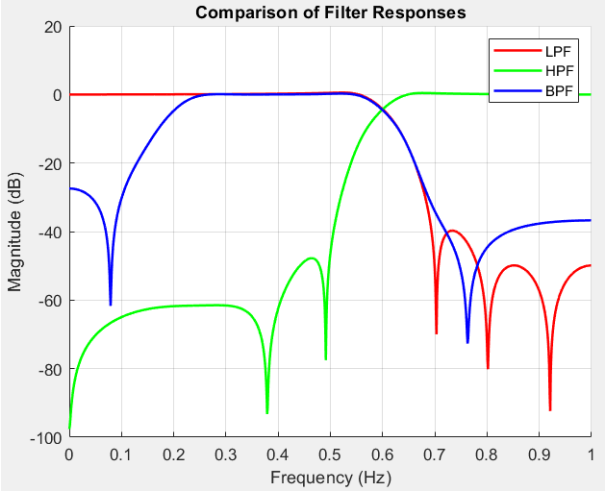












Penjelasan:

Pada program ini memakai pendekatan yang lebih fleksibel dengan metode yulewalk, di mana pengguna bisa menentukan sendiri bentuk respons frekuensi melalui vektor amplitudo. Filter yang dihasilkan sangat spesifik, sesuai dengan kebutuhan desain. Sama seperti pada program pertama, filter ini juga mencakup LPF, HPF, dan BPF, tapi kontrol terhadap bentuk transisinya lebih presisi. Program ini juga membandingkan respons ketiga filter dalam satu grafik untuk analisis yang lebih jelas.

Pertanyaan:

Amati bentuk respon frekuensi yang dihasilkan oleh metode prototipe filter analog pada percobaan A dengan metode direct design pada percobaan B! Apakah yang anda dapatkan? Jelaskan!

Saat membandingkan respons frekuensi dari metode prototipe analog (percobaan A) dan direct design (percobaan B), perbedaannya cukup jelas. Metode prototipe menghasilkan respons yang lebih mulus dan natural, terutama di daerah transisi, khas dari karakteristik filter Butterworth. Frekuensi cutoff-nya dihitung berdasarkan magnitude -3 dB, yang sering dianggap standar untuk filter analog. Sehingga respon fase lebih baik dan transisi frekuensinya lebih halus.

Sebaliknya, metode direct design memberikan hasil yang lebih tajam dan terkontrol, karena bentuk responsnya langsung diatur sesuai kebutuhan melalui vektor amplitudo. Ini membuat desainnya lebih fleksibel dan bisa disesuaikan dengan kebutuhan spesifik. Namun, akibatnya, responsnya terkadang terlihat lebih kaku di daerah transisi. Sehingga lebih fleksibel dan presisi dalam menentukan bentuk spektrum amplitude

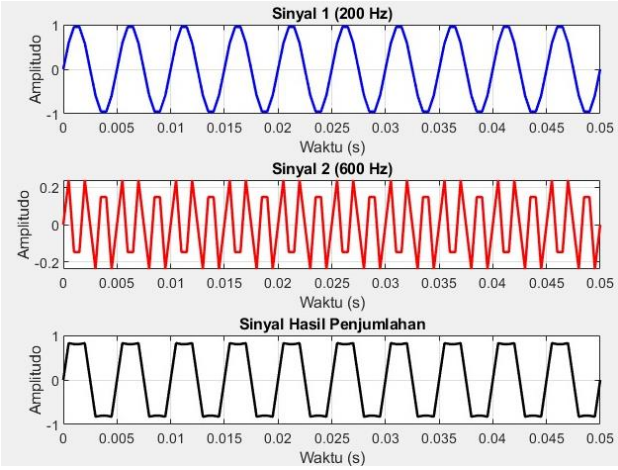
IIR Filter Sinyal Sederhana

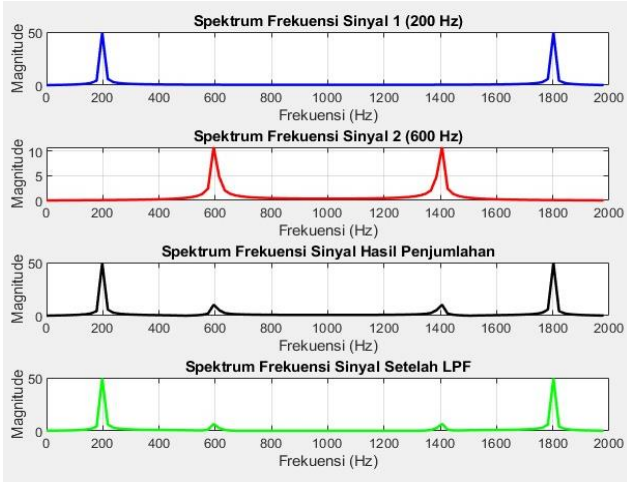
Program

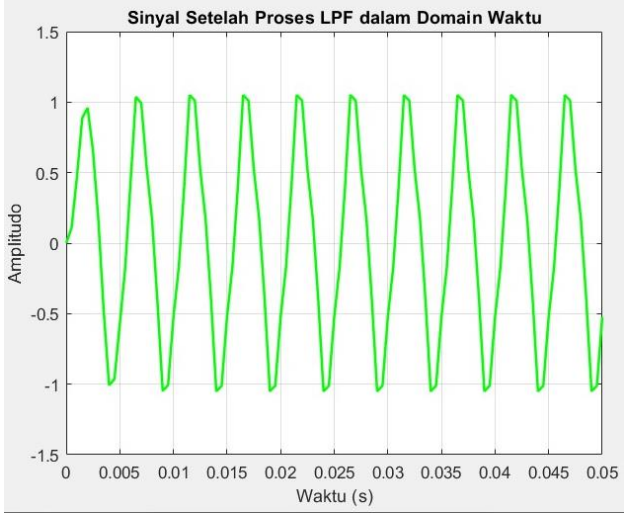
```
clc;
clear;
close all;

% Parameter
fs = 2000;          % Sampling rate (Hz)
t = 0:1/fs:0.05;    % Waktu (50 ms)
A1 = 1;             % Amplitudo sinyal pertama
f1 = 200;           % Frekuensi sinyal pertama (Hz)
A2 = 0.25;          % Amplitudo sinyal kedua
f2 = 600;           % Frekuensi sinyal kedua (Hz)

% 1. Bangkitkan sinyal sinus pertama
sinyal1 = A1 * sin(2 * pi * f1 * t);

% 2. Bangkitkan sinyal sinus kedua
sinyal2 = A2 * sin(2 * pi * f2 * t);

% Penjumlahan kedua sinyal
sinyal_out = sinyal1 + sinyal2;

% 3. Proses pemfilteran menggunakan LPF (Metode A/B)
% (Gunakan filter dari langkah A atau B, contoh menggunakan Metode B: yulewalk)
F = [0 0.15 0.25 0.35 0.45 0.55 0.65 0.75 0.85 1];
A = [1 1 1 1 1 1 0 0 0 0];
N = 10; % Orde filter
[b, a] = yulewalk(N, F, A); % Desain LPF menggunakan metode direct design
sinyal_filtered = filter(b, a, sinyal_out);

% 4. Plot domain waktu
figure;
subplot(3, 1, 1);
plot(t, sinyal1, 'b', 'LineWidth', 1.5);
title('Sinyal 1 (200 Hz)');
xlabel('Waktu (s)');
ylabel('Amplitudo');
grid on;

subplot(3, 1, 2);
plot(t, sinyal2, 'r', 'LineWidth', 1.5);
title('Sinyal 2 (600 Hz)');
```

```
xlabel('Waktu (s)');
ylabel('Amplitudo');
grid on;

subplot(3, 1, 3);
plot(t, sinyal_out, 'k', 'LineWidth', 1.5);
title('Sinyal Hasil Penjumlahan');
xlabel('Waktu (s)');
ylabel('Amplitudo');
grid on;

% 5. Plot domain frekuensi
n = length(t);
f = (0:n-1)*(fs/n); % Frekuensi

% FFT Sinyal
fft_sinyal1 = abs(fft(sinyal1));
fft_sinyal2 = abs(fft(sinyal2));
fft_sinyal_out = abs(fft(sinyal_out));
fft_sinyal_filtered = abs(fft(sinyal_filtered));

figure;
subplot(4, 1, 1);
plot(f, fft_sinyal1, 'b', 'LineWidth', 1.5);
title('Spektrum Frekuensi Sinyal 1 (200 Hz)');
xlabel('Frekuensi (Hz)');
ylabel('Magnitude');
grid on;

subplot(4, 1, 2);
plot(f, fft_sinyal2, 'r', 'LineWidth', 1.5);
title('Spektrum Frekuensi Sinyal 2 (600 Hz)');
xlabel('Frekuensi (Hz)');
ylabel('Magnitude');
grid on;

subplot(4, 1, 3);
plot(f, fft_sinyal_out, 'k', 'LineWidth', 1.5);
title('Spektrum Frekuensi Sinyal Hasil Penjumlahan');
xlabel('Frekuensi (Hz)');
ylabel('Magnitude');
grid on;

subplot(4, 1, 4);
plot(f, fft_sinyal_filtered, 'g', 'LineWidth', 1.5);
title('Spektrum Frekuensi Sinyal Setelah LPF');
xlabel('Frekuensi (Hz)');
ylabel('Magnitude');
grid on;

% 6. Plot domain waktu untuk sinyal setelah LPF
figure;
plot(t, sinyal_filtered, 'g', 'LineWidth', 1.5);
title('Sinyal Setelah Proses LPF dalam Domain Waktu');
xlabel('Waktu (s)');
ylabel('Amplitudo');
grid on;
```

Output:







Penjelasan:

Pada program ini merupakan pengaplikasian langsung filter terhadap sinyal gabungan dua frekuensi. Setelah filter diterapkan (dalam contoh ini menggunakan metode direct design), sinyal dengan frekuensi tertentu berhasil diisolasi. Visualisasi dalam domain waktu dan frekuensi membantu kita memahami bagaimana filter bekerja memisahkan komponen sinyal.

Diskusi

a. Bandingkan karakteristik filter hasil perancangan dengan analog prototipe Butterworth dan direct design Yule-Walker. Jelaskan perbedaannya!

Filter Butterworth dirancang untuk menghasilkan respons frekuensi yang halus tanpa adanya ripple. Transisinya lebih lembut, namun roll-off-nya relatif lambat, sehingga cocok untuk aplikasi yang membutuhkan fidelitas fase dan kehalusan, seperti audio dan komunikasi. Fase yang lebih linier membuatnya ideal untuk sinyal yang sensitif terhadap distorsi fase. Sebaliknya, filter Yule-Walker menawarkan kontrol yang lebih fleksibel terhadap magnitudo dengan roll-off yang lebih tajam, namun respons ini biasanya disertai ripple. Cutoff-nya lebih presisi dengan daerah transisi yang lebih sempit, tetapi fase yang kurang linier dapat menyebabkan distorsi pada sinyal tertentu.

Kesimpulan:
Butterworth lebih unggul untuk aplikasi yang membutuhkan kehalusan dan fidelitas fase tinggi, sedangkan Yule-Walker lebih cocok untuk aplikasi yang membutuhkan cutoff tajam dan kontrol magnitudo yang lebih fleksibel.

b. Apakah yang dapat anda simpulkan dari setiap hasil perbandingan sinyal input dan output dalam domain waktu dan domain frekuensi? Jelaskan!

Domain Waktu:

- *Butterworth*: Output lebih menyerupai sinyal input dengan frekuensi rendah terjaga.
- *Yule-Walker*: Ada sedikit distorsi amplitudo di daerah cutoff.
- Keduanya berhasil meredam frekuensi tinggi (600 Hz) dan mempertahankan frekuensi rendah (200 Hz).

Domain Frekuensi:

- *Butterworth*: Transisi lembut di cutoff, menjaga komponen penting.
- *Yule-Walker*: Cutoff lebih tajam, cepat meredam frekuensi tinggi.

Kesimpulan:
Butterworth adalah pilihan ideal untuk sinyal yang memprioritaskan kehalusan dan menjaga amplitudo atau fase, sementara Yule-Walker lebih unggul dalam aplikasi yang membutuhkan respons cutoff yang presisi.

c. Menurut anda, lebih efektif manakah antara FIR dan IIR filter? Untuk menjawab pertanyaan ini, anda diminta untuk membuat desain filter FIR dan IIR low pass untuk menyelesaikan kasus C diatas. Analisa meliputi, jumlah order N, tingkat kecuraman hasil filter, dsb.

KRITERIA	***FIR***	***IIR***
ORDER	Lebih tinggi	Lebih rendah
ROLL-OFF	Lambat	Curam
LINIERITAS FASE	Linier (ideal untuk audio)	Tidak linier
EFISIENSI	Kurang efisien	Sangat efisien
STABILITAS	Stabil	Rentan overshoot

FIR Filter: Ideal untuk aplikasi yang membutuhkan fase linier sempurna, seperti audio atau kontrol, meski membutuhkan order tinggi.

IIR Filter: Efisien dengan order rendah dan roll-off tajam, cocok untuk komunikasi digital.

Kesimpulan:
FIR cocok untuk aplikasi yang memprioritaskan linearitas fase, seperti pengolahan sinyal audio, sedangkan IIR lebih efektif untuk kebutuhan praktis yang mengutamakan efisiensi dan roll-off tajam, seperti komunikasi atau filtrasi sinyal berkecepatan tinggi.